

EN UNA FRASE Sabes proteger la API con Spring Security: decides qué rutas son públicas y cuáles exigen un rol, cargas los usuarios desde la base de datos y guardas las contraseñas como hash BCrypt. La autenticación es HTTP Basic sin estado, la base sobre la que se monta JWT en el capítulo siguiente.

LO ESENCIAL

<code>spring-boot-starter-security</code>	Al añadirlo, todos los endpoints quedan protegidos: usuario <code>user</code> y contraseña aleatoria que aparece en el log de arranque
<code>spring-security-test (scope test)</code>	Permite simular usuarios (<code>@WithMockUser(roles = "ADMIN")</code>) y tokens CSRF (<code>.with(csrf())</code>) en los tests de <code>@WebMvcTest</code> , que ahora llevan <code>@Import(SeguridadConfig.class)</code>
<code>SecurityFilterChain</code>	Bean central: rutas públicas, rutas protegidas, mecanismo de autenticación y CSRF
<code>@EnableWebSecurity</code>	Activa la configuración personalizada, en <code>SeguridadConfig</code> (paquete <code>configuracion</code>)
<code>.csrf(csrf → csrf.disable())</code>	Desactiva CSRF: en una API REST sin estado no aplica
<code>authorizeHttpRequests</code>	Reglas de acceso por ruta y método HTTP; van de la más específica a la más genérica
<code>permitAll() / authenticated()</code>	Ruta sin autenticación / ruta que exige credenciales válidas
<code>hasRole("ADMIN") / hasAnyRole("ADMIN", "USER")</code>	La ruta exige ese rol (<code>ROLE_ADMIN</code>) o cualquiera de los indicados
<code>.httpBasic(basic → {})</code>	Activa HTTP Basic: <code>Authorization: Basic base64(usuario:contraseña)</code> en cada petición
<code>SessionCreationPolicy.STATELESS</code>	Sin sesión HTTP: cada petición lleva sus propias credenciales
<code>frameOptions(frame → frame.sameOrigin())</code>	Deja que la consola H2, que usa un iframe, siga funcionando
<code>UserDetailsService</code>	Interfaz con un único método, <code>loadUserByUsername</code> , que devuelve un <code>UserDetails</code> (nombre, hash y roles)
<code>BCryptPasswordEncoder</code>	<code>encode()</code> genera un hash lento y con sal aleatoria; <code>matches()</code> compara texto plano con hash
<code>DaoAuthenticationProvider</code>	Conecta el <code>UserDetailsService</code> (carga el usuario) con el <code>PasswordEncoder</code> (verifica la contraseña)

ERRORES TÍPICOS

1 401 Unauthorized con usuario y contraseña correctos

✓ en `data.sql` hay una contraseña en texto plano. Guarda siempre el hash: `new BCryptPasswordEncoder().encode("admin123")`.

2 403 Forbidden en POST, PUT, PATCH y DELETE con credenciales válidas

✓ CSRF sigue activo. Añade `.csrf(csrf → csrf.disable())`.

3 La consola H2 sale en blanco o da 403

✓ faltan las dos cosas: `requestMatchers("/h2-console/**").permitAll()` y `frameOptions(frame → frame.sameOrigin())`.

4 NoSuchBeanDefinitionException de UserDetailsService al arrancar

✓ a `BibliotecaUserDetailsService` le falta `@Service` o está fuera de `com.javaesencial.biblioteca`.

5 El formulario /login redirige en bucle con STATELESS

✓ es lo esperado: sin sesión no hay login por formulario. En una API REST las credenciales viajan en la cabecera de cada petición.

EN EL PROYECTO BIBLIOTECA

Entran `Credencial` y `CredencialJpaRepository` (cuenta de acceso a la API, independiente de `Usuario`), `BibliotecaUserDetailsService` en el paquete nuevo `seguridad` y `SeguridadConfig` en `configuracion`. La migración `V3__crear_tabla_credencial.sql` crea la tabla y `data.sql` inserta `admin` (rol `ADMIN`, contraseña `admin123`) y `bibliotecario` (rol `USER`, contraseña `user123`) con sus hashes BCrypt. `LibroControllerTest` se adapta a la seguridad. Quedan pendientes JWT (Capítulo 14) y Actuator + Docker (Capítulo 15).

■ CÓDIGO MÍNIMO Y PETICIONES

Las reglas de `SeguridadConfig`: lectura pública, escritura solo para `ADMIN`, préstamos para `ADMIN` o `USER`.

```
http
    .csrf(csrf → csrf.disable())
    .sessionManagement(session →
        session.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
    .headers(headers → headers.frameOptions(frame → frame.sameOrigin()))
    .authorizeHttpRequests(auth → auth
        .requestMatchers(HttpMethod.GET, "/libros", "/libros/**").permitAll()
        .requestMatchers("/docs/**", "/v3/api-docs/**", "/h2-console/**").permitAll()
        .requestMatchers(HttpMethod.POST, "/libros").hasRole("ADMIN")
        .requestMatchers(HttpMethod.PUT, "/libros/**").hasRole("ADMIN")
        .requestMatchers(HttpMethod.DELETE, "/libros/**").hasRole("ADMIN")
        .requestMatchers("/prestamos/**").hasAnyRole("ADMIN", "USER")
        .anyRequest().authenticated())
    .httpBasic(basic → {
    });
```

JAVA

`BibliotecaUserDetailsService` traduce una `Credencial` a `UserDetails`. El rol se guarda sin prefijo (`ADMIN`, `USER`): `roles(...)` añade `ROLE_` por ti.

```
@Override
public UserDetails loadUserByUsername(String username) {
    Credencial credencial = credenciales.findByUsername(username)
        .orElseThrow(() → new UsernameNotFoundException("Usuario no encontrado: " + username));

    return User.builder()
        .username(credencial.getUsername())
        .password(credencial.getPassword())
        .roles(credencial.getRol())
        .build();
}
```

JAVA

```
### GET público, sin credenciales
GET http://localhost:8080/libros

### POST protegido con credenciales correctas → 201 Created
POST http://localhost:8080/libros
Content-Type: application/json
Authorization: Basic YWRtaW46YWRtaW4xMjM=

{"titulo": "El Quijote", "autor": "Cervantes", "anio": 1605}

### En IntelliJ puedes escribir usuario y contraseña sin codificar
POST http://localhost:8080/libros
Content-Type: application/json
Authorization: Basic admin admin123

{"titulo": "El Quijote", "autor": "Cervantes", "anio": 1605}
```

HTTP